A Hybrid Classical-Quantum Chess Move-Selection Prototype: Design and Simulator Evaluation

Team 1
Amrita Vishwa Vidyapeetham, Section A

ABSTRACT

This paper describes an educational chess engine that combines classical chess logic with quantum-circuit demonstrations. It covers an 8x8 hybrid system using Grover search on indices of classically ranked moves; a 3x3 comparison of depth-3 minimax, a small-board circuit, and a hybrid selector; and a 2x2 gate-tracing example. A four-position seeded benchmark used Qiskit Aer with noise disabled. Mean execution times were 1.19 ms (classical), 95.81 ms (pure circuit), and 71.74 ms (hybrid). Classical minimax used the least measured time and memory, while the hybrid circuit used fewer qubits and gates than the pure-circuit path. The workloads differ, legal moves and oracle targets are prepared classically, and the transition circuit is incomplete. These measurements do not establish quantum speedup or compare playing strength.

Keywords: chess engines, Grover search, minimax, hybrid quantum systems, Qiskit Aer

I. INTRODUCTION

Chess programs combine legal-move generation, position evaluation, and search over possible continuations. Classical game-tree methods such as minimax and alpha-beta pruning provide a well-established baseline for this structure [1], [2]. Quantum search offers a different primitive: for a search space of N items and M marked items, amplitude amplification can raise the probability of observing a marked item using on the order of sqrt(N/M) oracle queries under the usual oracle model [3]-[5]. That query bound does not by itself make a chess engine faster. The predicate must be reversible, and the cost of preparing candidates, evaluating them, and implementing the oracle must be included.

This project studies that boundary with three small, inspectable systems. The 8x8 implementation provides a hybrid move-index search. The 3x3 board supports a controlled comparison of classical selection and two circuit workloads. The 2x2 board makes coordinate and gate behavior concrete. In each case, the classical board state remains authoritative. We therefore describe the work as a classical chess engine with quantum circuit demonstrations, rather than a complete quantum chess engine.

II. RELATED WORK

Shannon's early formulation of computer chess identifies move search and position evaluation as core engineering problems [1]. Knuth and Moore analyze alpha-beta pruning as a method for reducing the game tree explored by minimax [2]; the project uses this classical family for its depth-limited baseline. These methods search continuations in a game tree, whereas Grover search locates marked elements in a finite search space. They solve related selection tasks but are not interchangeable algorithms.

Grover introduced amplitude amplification for unstructured search, and later analyses addressed multiple marked items and iteration counts [3]-[5]. Quantum algorithms for evaluating NAND and minimax trees address game-tree evaluation more directly, but rely on different models and assumptions from a small Grover index circuit [6], [7]. Cantwell's quantum-chess work defines a chess variant whose rules use quantum behavior; it differs from this project's classical chess rules plus quantum search and circuit demonstrations [8]. Preskill discusses practical constraints of noisy intermediate-scale quantum devices [9]. LaRose surveys gate-level software platforms, and Qiskit Aer documentation describes the simulator used here [10], [12]. Standard circuit notation and state-vector concepts follow Nielsen and Chuang [11].

III. LITERATURE GAPS IDENTIFIED

The literature reviewed here supplies classical chess-search methods, quantum unstructured search, and quantum game-tree algorithms, but these do not provide a ready-made, end-to-end reversible oracle for the project's chess move representation. Legal-move generation, reversible scoring of chess positions, and complete board-state updates must be designed before Grover can search chess moves coherently. The distinction between quantum chess as a rule variant and quantum search applied to classical chess move lists also needs to be explicit. Simulator resource counts alone cannot establish practical speedup: workload equivalence, oracle cost, noise, measurement, and move quality must be evaluated. These gaps motivate an implementation study focused on transparent data flow and bounded claims rather than an advantage claim.

IV. WHY QUANTUM AND PROJECT CONTRIBUTIONS

Quantum search is relevant as a way to explore whether a list of move candidates can be represented by indices, marked by a predicate, and sampled after amplitude amplification. The project's present oracle receives target indices selected by a classical heuristic, so this is an educational integration experiment rather than a quantum replacement for minimax.

- Implements and documents three board-scale demonstrations: 2x2 gate tracing, 3x3 classical and circuit comparison, and 8x8 hybrid move-index search.
- Connects classical legal-move generation and heuristic ranking to a Qiskit Grover circuit, while separately exposing coordinate/status encoding and controlled-gate behavior in the small-board prototype.
- Reports simulator timing and circuit-resource measurements for four seeded 3x3 positions, with limitations that prevent speedup or playing-strength claims.

V. METHODOLOGY

A. Implemented Systems and Data Flow

8x8 hybrid system. The classical move generator creates legal moves from the board. A heuristic scores capture value, pawn advancement, and center preference. Python collects all indices tied for the highest score. Grover prepares a uniform superposition over a padded binary index space, phase-marks those indices, applies a diffuser, and measures an index. Classical code decodes and validates the result, then applies the move to the classical game state. A depth-3 minimax fallback is available. The candidate list and marked set are prepared classically.

3x3 systems. The classical baseline selects a move using depth-3 minimax with alpha-beta pruning and heuristic move ordering. The pure-circuit route receives a predetermined legal move, loads its coordinates and square statuses into a 23-qubit circuit, runs limited direction/status logic, and measures the registers. It demonstrates circuit construction and measurement; it does not autonomously choose the move or update the board. The hybrid comparison generates legal moves and scores them classically, then simulates Grover over top-scoring indices. The normal 3x3 game route uses classical minimax; the hybrid comparison is a separate evaluation path.

2x2 system. A single Black Pawn at (0,0) is used to explain how source and target coordinates, status lookups, and controlled gates are represented. For example, (0,0) to (1,0) is encoded in the small-board registers. This is a gate-level demonstration, not a complete pawn-rule implementation or quantum board transition. Grover's move-index circuit is separate and is used only when the candidate-search path has a nontrivial set of choices.

The three-system flow diagram is included in Appendix Fig. A. It separates classical preparation, circuit stages, measurement, and classical board updates.

B. Quantum Encoding and Grover Circuit

For the small-board circuit, a coordinate uses two bits for column and two for row. A 2x2 source at (0,0) maps to 0000; target (1,0) maps to 0001 under the project's logical column-then-row convention. The circuit allocates source and target coordinates (4 qubits each), direction workspace (4), source and destination status (3 each), and ancilla workspace (5), for 23 qubits total. Status values include empty 000 and Black Pawn 001. X gates prepare one-bits and convert zero-controls into positive controls; CX and CCX contribute to limited direction logic; multi-controlled X gates implement coordinate-conditioned status lookups and attempted status operations. The actual board transition remains classical.

For Grover, n=ceil(log2 C) index qubits represent C candidates in a padded space of 2^n. Hadamard gates prepare the uniform state. The flag is prepared in |->; the oracle's multi-controlled X produces phase kickback, multiplying marked-index amplitudes by -1. The diffuser D=2|s><s|-I reflects amplitudes about their mean. The oracle/diffuser pair repeats for an iteration count estimated from the search-space and target counts. Measurement returns a bit string that classical code decodes to a move-list index. The oracle does not compute chess legality or heuristic scores coherently: those are computed before circuit construction.

C. Benchmark Design and Configuration

The comparison uses four unique 3x3 positions generated deterministically from authored piece layouts and seeded legal random walks (seed 20261013). There is no external training dataset and no learned model, so train/test splitting is not applicable. The classical selector is depth-3 minimax; the pure-circuit input is the first legal move in each position and uses one shot; the hybrid path scores the candidate list classically and runs Grover with 2048 shots. Both quantum paths use Qiskit Aer on CPU with the matrix-product-state method and noise disabled. Simulator initialization is excluded from per-position timing. These settings are specific to this comparison; the shared application wrapper defaults to 1024 shots and an illustrative depolarizing noise model.

Metrics are mean elapsed time per position; mean additional process resident memory above the pre-step baseline, sampled every 2 ms; average legal-move count; qubit count; gate count; and circuit depth. Memory sampling is approximate and may miss short peaks. Timing covers move selection for classical minimax, circuit construction plus simulation for the pure route, and candidate generation/scoring plus Grover construction, transpilation, and simulation for the hybrid route. This is a small engineering benchmark, not a statistically powered or hardware comparison.

VI. RESULTS AND ANALYSIS

Table I summarizes the four-position means; Appendix Fig. B shows the resource panels. Classical depth-3 minimax had the lowest measured time (1.19 ms) and additional peak memory (about 0.01 MB). It builds no circuit, so circuit metrics are not applicable. The pure-circuit demonstration had the largest measured simulator cost: 95.81 ms and 5.19 MB, with 23 qubits, 102 gates, and mean depth 32.25. The hybrid path averaged 71.74 ms and 0.04 MB, with 2.25 qubits, 13.75 gates, and mean depth 10.25. Its runtime was approximately 25.1% lower than the pure-circuit route, and its measured circuit used fewer qubits and gates.

These values must be interpreted by workload. The pure circuit processes a specified move; the hybrid path ranks and searches candidates; minimax explores a shallow game tree. The raw times therefore do not compare equivalent tasks. Hybrid simulation remained slower than the classical baseline, and the marked set was prepared with classical scoring. The simulator experiment does not establish quantum speedup, improved move quality, or performance on quantum hardware.

TABLE I. MEAN RESULTS FOR FOUR SEEDED 3x3 POSITIONS

System	Time (ms)	Extra RAM (MB)	Legal moves
Classical minimax	1.19	0.0068	2.25
Pure circuit	95.81	5.1924	2.25
Hybrid Grover	71.74	0.0430	2.25

Structural circuit metrics: Classical minimax: not applicable (no circuit); pure circuit: 23 qubits, 102 gates, depth 32.25; hybrid: mean 2.25 qubits, 13.75 gates, depth 10.25. Mean values are taken from the saved benchmark summary. RAM is additional peak resident memory above baseline, sampled every 2 ms.

VII. CONCLUSION AND OUTLOOK

The project provides a transparent bridge between classical chess logic and quantum-circuit concepts. Its strongest implemented quantum-search example is the 8x8 hybrid pipeline: legal moves and heuristic targets are prepared classically, then a Grover circuit amplifies selected indices. The 2x2 and 3x3 circuits help inspect gate-level encodings but remain prototypes. In the measured 3x3 run, classical minimax had the smallest runtime and memory cost; the hybrid Grover circuit was structurally smaller than the pure-circuit demonstration but did not outperform the classical baseline.

Next work should define an injective encoding for every piece and empty square, implement and verify reversible legality and scoring oracles, and uncompute temporary data. The small-board transition should be checked against truth tables before adding complete board updates. Experiments should include a broader position set, repeated runs with uncertainty estimates, equivalent end-to-end workloads, explicit oracle costs, and separate noise-on and hardware measurements. Move quality should be measured against a specified classical reference. Until then, results should be described as simulator measurements of a hybrid prototype.

APPENDIX A. SUPPORTING CIRCUIT MATERIAL

A. Small-board register allocation

The 23-qubit register layout is cur(4) + tgt(4) + dir(4) + src(3) + dst(3) + anc(5). The worked 2x2 example begins with cur=0000, tgt=0001, src=001 (Black Pawn), and dst=000 (empty). In the current prototype, the direction comparator reports dir=0000 for this forward move; therefore the conditional status operator does not fire. The measurement is diagnostic, and the classical game engine performs any board update.

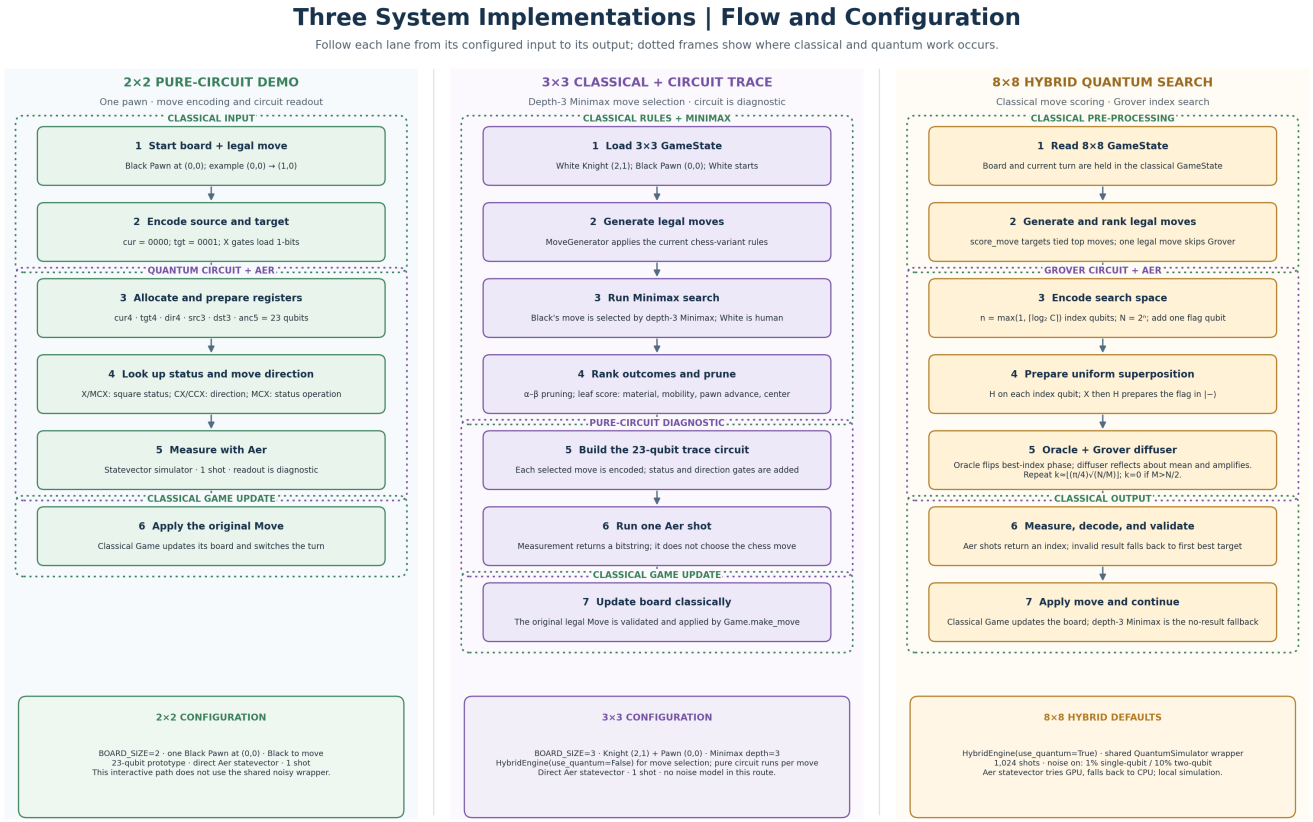
APPENDIX B. HYBRID MOVE-INDEX SEARCH

The following pseudocode summarizes the implemented classical preparation and quantum sampling boundary. The index oracle receives already selected target indices; it does not calculate chess legality or heuristic values inside the circuit.

```
legal_moves = generate_legal_moves(classical_state)
scores = [heuristic(state, move) for move in legal_moves]
targets = indices_with_maximum_score(scores)
if every candidate is marked: return first target
q = max(1, ceil(log2(len(legal_moves))))
prepare H^q uniform index state and |-> flag
repeat Grover oracle phase mark + diffuser
measure index; decode and validate classically
```

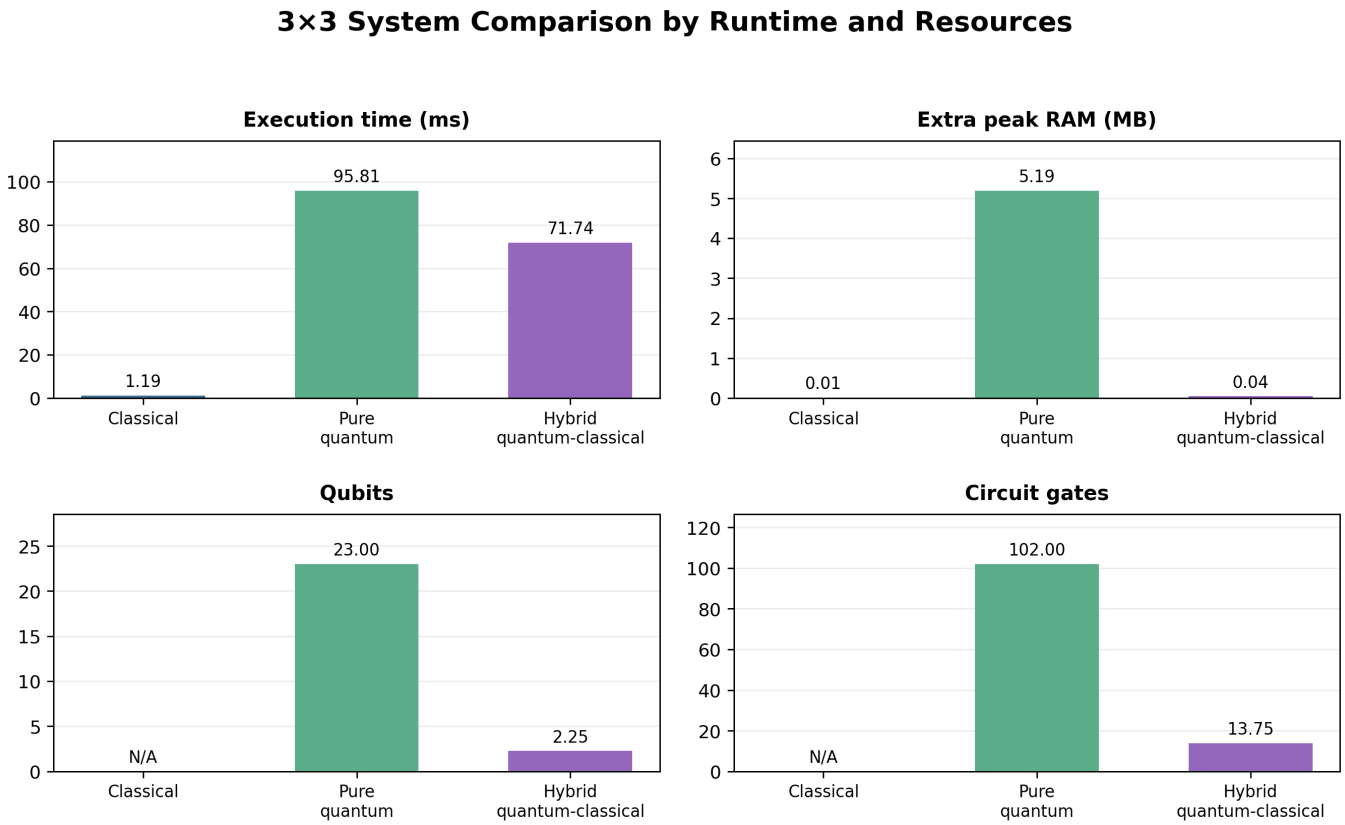
Implementation files: ai/quantum_search.py, quantum/grover.py, quantum/oracle.py, quantum/diffuser.py, quantum/simulator.py, and pure_quantum_engine/circuit.py. The diagrams document data flow and wiring; they do not imply a complete reversible board transition.

FIGURE A. THREE SYSTEM IMPLEMENTATION FLOWS



The dotted frames distinguish classical preparation, circuit work, and classical outputs. The 2x2, 3x3, and 8x8 lanes use different circuits and configurations.

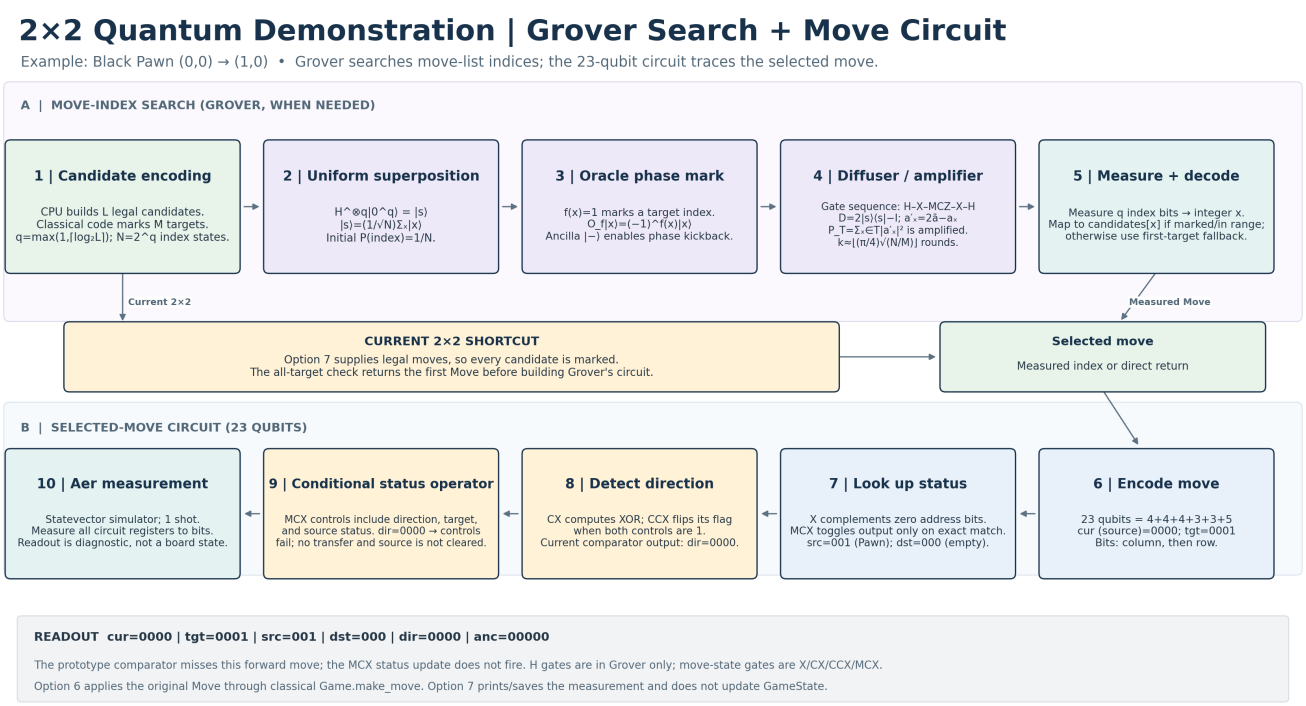
FIGURE B. 3x3 BENCHMARK RESOURCE COMPARISON



Same four seeded positions; mean values. Extra peak RAM is sampled above baseline. N/A means no quantum circuit.

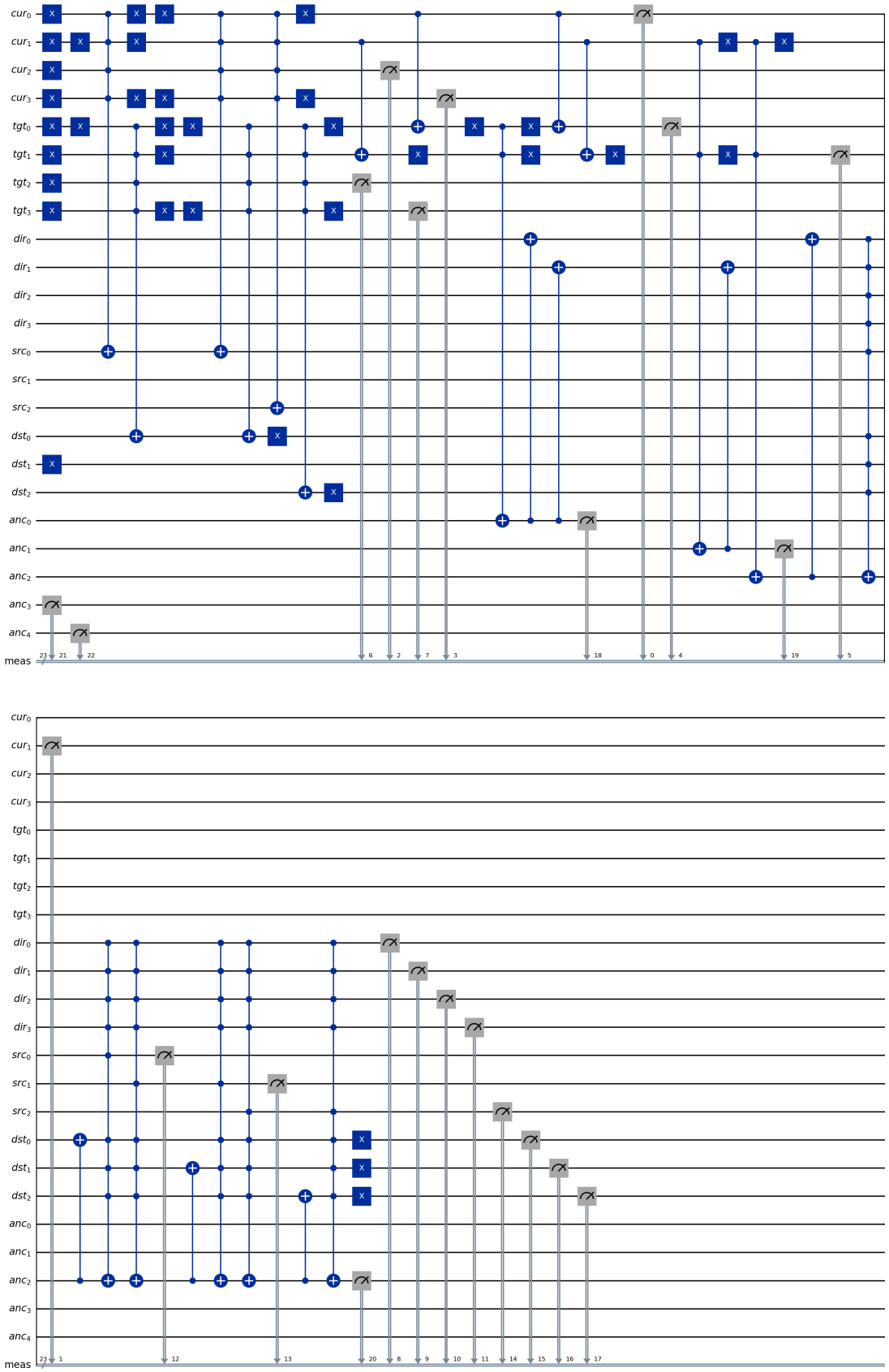
Mean measurements over four seeded positions. N/A denotes the classical system, which constructs no quantum circuit. The systems perform different operations, so these values describe measured resource use rather than relative move quality or quantum speedup.

FIGURE C. 2x2 GROVER AND SELECTED-MOVE CIRCUIT FLOW



For this single-candidate 2x2 example, the current all-target shortcut returns the move without constructing Grover's circuit. The 23-qubit circuit is a separate diagnostic move trace.

FIGURE D. SMALL-BOARD CIRCUIT DIAGRAM



The circuit encodes coordinates, performs board-status lookup gates, attempts direction/status logic, and measures the registers. The board update remains classical.

REFERENCES

[1] C. E. Shannon, “XXII. Programming a Computer for Playing Chess,” The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science, vol. 41, no. 314, pp. 256-275, 1950. doi: 10.1080/14786445008521796.

[2] D. E. Knuth and R. W. Moore, “An Analysis of Alpha-Beta Pruning,” Artificial Intelligence, vol. 6, no. 4, pp. 293-326, 1975. doi: 10.1016/0004-3702(75)90019-3.

[3] L. K. Grover, “A Fast Quantum Mechanical Algorithm for Database Search,” in Proc. 28th ACM Symposium on Theory of Computing, 1996, pp. 212-219. doi: 10.1145/237814.237866.

[4] M. Boyer, G. Brassard, P. Hoyer, and A. Tapp, “Tight Bounds on Quantum Searching,” Fortschritte der Physik, vol. 46, nos. 4-5, pp. 493-505, 1998. doi: 10.1002/(SICI)1521-3978(199806)46:4/5<493::AID-PROP493>3.0.CO;2-P.

[5] G. Brassard, P. Hoyer, M. Mosca, and A. Tapp, “Quantum Amplitude Amplification and Estimation,” Contemporary Mathematics, vol. 305, pp. 53-74, 2002. doi: 10.1090/conm/305/05215.

[6] E. Farhi, J. Goldstone, and S. Gutmann, “A Quantum Algorithm for the Hamiltonian NAND Tree,” Theory of Computing, vol. 4, pp. 169-190, 2008. doi: 10.4086/toc.2008.v004a008.

[7] R. Cleve, D. Gavinsky, and D. L. Yonge-Mallo, “Quantum Algorithms for Evaluating Min-Max Trees,” in Theory of Quantum Computation, Communication, and Cryptography, LNCS 5106, 2008, pp. 11-15. doi: 10.1007/978-3-540-89304-2_2.

[8] C. Cantwell, “Quantum Chess: Developing a Mathematical Framework and Design Methodology for Creating Quantum Games,” arXiv:1906.05836, 2019.

[9] J. Preskill, “Quantum Computing in the NISQ Era and Beyond,” Quantum, vol. 2, p. 79, 2018. doi: 10.22331/q-2018-08-06-79.

[10] R. LaRose, “Overview and Comparison of Gate Level Quantum Software Platforms,” Quantum, vol. 3, p. 130, 2019. doi: 10.22331/q-2019-03-25-130.

[11] M. A. Nielsen and I. L. Chuang, Quantum Computation and Quantum Information, 10th Anniversary ed. Cambridge University Press, 2010.

[12] Qiskit Aer Developers, “Qiskit Aer Documentation.” Accessed: Oct. 15, 2026. [Online]. Available: https://qiskit.github.io/qiskit-aer/